



Instituto Tecnológico y de Estudios Superiores de Monterrey

Tecnológico de Monterrey. Campus Monterrey

Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales [ReflexEvidencia1]

A01287002 – Andrés Rodríguez Cantú

Profesor: David Alonso Cantú Delgado

Programación de estructuras de datos y algoritmos fundamentales (Gpo 607)

Monterrey, Nuevo León

Domingo 20 de septiembre, 2026

Importancia de ordenar/buscar en este tipo de logs.

Tener ordenados los logs permite realizar una búsqueda extremadamente efectiva y rápida. Utilizando algoritmos como el de búsqueda binaria hace que una tarea de organización se transforme en una tarea de búsqueda con mucha eficiencia. Incluso aunque búsqueda binaria no sea lo más eficiente, tener los datos organizados transforma tu problema de $O(n)$ a $O(\log n)$; representando de segundos a milisegundos de tiempo de ejecución.

Comparación conceptual de los algoritmos elegidos (fortalezas, debilidades y casos de uso), incluyendo por qué uno de ellos se comporta distinto en datos casi ordenados.

Quicksort generalmente fue el más rápido para el caso 1, sin embargo para el caso 2, este siempre fue el más lento. El pivote hizo que el algoritmo se comportara de manera ineficiente, mientras que Mergesort y Heapsort se comportaron de manera más estable. En el caso 3, Introsort fue el más rápido, ya que combina la eficiencia de Quicksort con la estabilidad de Heapsort.

Interpretación de sus mediciones, contrastadas con su predicción inicial y con la complejidad teórica.

Uso de IA: 2-3 prompts clave que utilizaron, qué modificaron del código sugerido, y — obligatorio — al menos un caso concreto en el que la IA se equivocó o sugirió algo algorítmicamente incorrecto o ineficiente (por ejemplo, búsqueda lineal en vez de binaria, un ordenamiento inestable donde importaba estabilidad, un error de límites en el rango de fechas) y cómo lo detectaron y corrigieron.

Primer prompt

INPUT

can I force it to write with a format this definitions?

```
#define printDayTime(dt) \  
    +dt.year << "-" << +dt.month << "-" << +dt.day << " " << +dt.hour  
<< ":" << +dt.minute << ":" << +dt.second
```

So that rather than `2026-9-7 23:8:26`, it shows `2026-09-07 23:08:26`

OUTPUT

```
#include <format>  
  
#define printDayTime(dt) \  
    std::format("{:04}-{:02}-{:02} {:02}:{:02}:{:02}", \  
        +dt.year, +dt.month, +dt.day, +dt.hour, +dt.minute,  
        +dt.second)
```

Justificación: Se utilizó la librería `<format>` para formatear la salida de la fecha y hora, asegurando que los valores se muestren con ceros a la izquierda cuando sea necesario, gracias a esto, se logró no solo la impresión, sino también la colocación de ceros a la izquierda en los valores de mes, día, hora, minuto y segundo.

Segundo prompt

INPUT

The purpose of this inquiry is to attempt to create a struct class that allows me to self correct IPV4 errors. How could one do that with this setup?

```
typedef struct {
    uint8_t a;
    uint8_t b;
    uint8_t c;
    uint8_t d;
} IPAddress;

typedef struct __attribute__((packed)) {
    IPAddress ip;
    uint8_t sec_ded;
} IPAddressSEC_DED;
```

OUTPUT

```
// 32 unique weight-3 patterns for a 7-bit parity matrix
static const uint8_t P_MATRIX[32] = {
    0x07, 0x0B, 0x0D, 0x0E, 0x13, 0x15, 0x16, 0x19,
    0x1A, 0x1C, 0x23, 0x25, 0x26, 0x29, 0x2A, 0x2C,
    0x31, 0x32, 0x34, 0x43, 0x45, 0x46, 0x49, 0x4A,
    0x4C, 0x51, 0x52, 0x54, 0x61, 0x62, 0x64, 0x70
};

static inline uint32_t ip_to_u32(IPAddress ip) {
    return ((uint32_t)ip.a << 24) | ((uint32_t)ip.b << 16) |
           ((uint32_t)ip.c << 8) | (uint32_t)ip.d;
}
```

```

void encode_ip(IPAddressSEC_DED *packet) {
    uint32_t val = ip_to_u32(packet->ip);
    uint8_t parity = 0;
    for (int i = 0; i < 32; i++) {
        if ((val >> i) & 1) parity ^= P_MATRIX[i];
    }
    packet->sec_ded = parity;
}

typedef enum { ERROR_NONE, ERROR_CORRECTED_DATA,
ERROR_CORRECTED_PARITY, ERROR_DOUBLE } Status;

Status verify_and_correct(IPAddressSEC_DED *packet) {
    uint32_t val = ip_to_u32(packet->ip);
    uint8_t calc_parity = 0;
    for (int i = 0; i < 32; i++) {
        if ((val >> i) & 1) calc_parity ^= P_MATRIX[i];
    }

    uint8_t syndrome = (calc_parity ^ packet->sec_ded) & 0x7F;
    if (syndrome == 0) return ERROR_NONE;

    // Search for matching data bit error (Weight 3)
    for (int i = 0; i < 32; i++) {
        if (syndrome == P_MATRIX[i]) {
            val ^= (1U << i); // Correct corrupted bit
            packet->ip.a = (val >> 24) & 0xFF;
            packet->ip.b = (val >> 16) & 0xFF;
            packet->ip.c = (val >> 8) & 0xFF;
            packet->ip.d = val & 0xFF;
            return ERROR_CORRECTED_DATA;
        }
    }

    // Single-bit parity error (Weight 1)
    if (__builtin_popcount(syndrome) == 1) {
        packet->sec_ded ^= syndrome;
        return ERROR_CORRECTED_PARITY;
    }
}

```

```
// Even weight or unmapped syndrome indicates double-bit error
return ERROR_DOUBLE;
}
```

Justificación: Se implementó un esquema de codificación y corrección de errores basado en un código de Hamming extendido para detectar y corregir errores en direcciones IPv4 (que luego se utilizó para las fechas). De esta manera, en redes, se puede detectar y corregir errores de un solo bit en la dirección IP, y también detectar errores de dos bits, lo que mejora la confiabilidad de la transmisión de datos.

Error de la IA

Mejoramiento de la máquina de estados: La IA sugirió un diagrama de flujo que no contemplaba la verificación de errores en la entrada del usuario, lo que podría llevar a fallos en la ejecución del programa. Realmente solo proponía mas overhead y cada vez que veía algo que no le gustaba, terminaba el programa con error. Se utilizó parte del diagrama, pero al final se implementó un manejo de errores más justo para el usuario haciendo uso con el do/while.

